

GATE Data Science & Artificial Intelligence (DA)

Question Bank: Dict/Set Complexity, Hashing Patterns & A^* Search

10 Selected PYQs (GATE DA / CS) + 10 Targeted GATE-Level Practice Questions

Topic Coverage:

1. **Topic 1:** Dictionary and Set Complexity Nuances (Collisions, Worst-Case vs Average-Case Lookups, Load Factor)
2. **Topic 2:** Hashing Patterns (Anagram Grouping, Deduplication Variants, Sliding Window Hashing)
3. **Topic 3:** Informed Search: A^* Mechanics (Evaluation Function $f = g + h$, Node Selection, Re-opening, Closed Sets)
4. **Topic 4:** Tracing A^* Graph Search Expansion on a 5-Node Graph

Part I: 10 Previous Year Questions (PYQs)

Q1. Expected Probes in Open Addressing Under Uniform Hashing [GATE DA 2024 • MCQ

• 1 Mark] Consider performing uniform hashing on an open-address hash table with load factor $\alpha < 1$, where n elements are stored in a table with m slots. The expected number of probes in an unsuccessful search is at most $\frac{1}{1-\alpha}$. What is the upper bound on the expected number of probes required to insert a new element into this hash table?

- (A) $\frac{1}{1-\alpha}$
- (B) $\frac{1}{\alpha} \ln \left(\frac{1}{1-\alpha} \right)$
- (C) $\frac{1}{1+\alpha}$
- (D) $1 + \frac{\alpha}{2}$

Answer: (A)

Solution / Explanation:

In open addressing, an insertion operation probes unoccupied slots following the exact same sequence as an unsuccessful search until an empty slot is encountered. Therefore, under the uniform hashing assumption, inserting an element requires at most the same expected number of probes as an unsuccessful search:

$$E[\text{probes to insert}] \leq \frac{1}{1-\alpha}.$$

(Option B corresponds to the average probes in a successful search).

Q2. Admissible Combination of Heuristics in A^* Search

[GATE DA 2024 •

MCQ • 1 Mark] Let $h_1(n)$ and $h_2(n)$ be two admissible heuristics used in A^* search for finding the shortest path to a goal. Which ONE of the following expressions is ALWAYS guaranteed to be an admissible heuristic?

- (A) $h_1(n) + h_2(n)$
- (B) $h_1(n) \times h_2(n)$

(C) $\frac{h_1(n)+h_2(n)}{2}$

(D) $(h_1(n))^2$

Answer: (C)

Solution / Explanation:

By definition, an admissible heuristic never overestimates the true remaining cost $h^*(n)$, so $0 \leq h_1(n) \leq h^*(n)$ and $0 \leq h_2(n) \leq h^*(n)$.

- For (C): $\frac{h_1(n)+h_2(n)}{2} \leq \frac{h^*(n)+h^*(n)}{2} = h^*(n)$. Hence, the arithmetic average is always admissible.
- For (A): $h_1(n) + h_2(n)$ can equal $2h^*(n) > h^*(n)$, which overestimates the true cost.
- For (B) and (D): Products and squares can easily exceed $h^*(n)$ when values are > 1 .

Q3. Open Addressing Linear Probing Slot Placement [GATE DA 2025 • MCQ • 1 Mark] Consider a hash table of size 10 with indices $\{0, 1, \dots, 9\}$, with the hash function $h(x) = (3x) \bmod 10$, where linear probing is used to handle collisions. The hash table is initially empty, and the sequence of keys 1, 4, 5, 6, 14, 15 is inserted into the hash table. The indices where the keys 14 and 15 are stored are, respectively:

(A) 2 and 5

(B) 2 and 6

(C) 4 and 5

(D) 4 and 6

Answer: (D)

Solution / Explanation:

Let us trace insertion into table $T[0 \dots 9]$:

- Insert 1: $h(1) = 3 \bmod 10 = 3 \implies T[3] = 1$.
- Insert 4: $h(4) = 12 \bmod 10 = 2 \implies T[2] = 4$.
- Insert 5: $h(5) = 15 \bmod 10 = 5 \implies T[5] = 5$.
- Insert 6: $h(6) = 18 \bmod 10 = 8 \implies T[8] = 6$.
- Insert 14: $h(14) = 42 \bmod 10 = 2$. Slot 2 is occupied (4). Probe index 3: occupied (1). Probe index 4: empty $\implies T[4] = 14$.
- Insert 15: $h(15) = 45 \bmod 10 = 5$. Slot 5 is occupied (5). Probe index 6: empty $\implies T[6] = 15$.

Keys 14 and 15 are stored at indices 4 and 6, respectively.

Q4. Worst-Case Lookup in Chaining vs Open Addressing [GATE CS 2019 • MCQ • 1 Mark]

Suppose n keys are inserted into an initially empty hash table of size m ($n \leq m$). In the worst case where all n keys produce identical hash values, what are the time complexities of searching for a key using Separate Chaining and Linear Probing, respectively?

(A) $\Theta(1)$ and $\Theta(1)$

(B) $\Theta(n)$ and $\Theta(n)$

(C) $\Theta(n)$ and $\Theta(1)$

(D) $\Theta(\log n)$ and $\Theta(n)$

Answer: (B)

Solution / Explanation:

In the adversarial worst-case scenario:

- In Separate Chaining: All n elements reside in a single linked list at bucket $h(k)$. Searching requires traversing the list of length n , taking $\Theta(n)$ time.
- In Linear Probing: All n keys collide and form a contiguous primary cluster of length n . Searching for a key requires probing every occupied slot in the cluster, taking $\Theta(n)$ comparisons.

Thus, both methods degrade to $\Theta(n)$ time in the worst case.

Q5. Optimal String Anagram Grouping Complexity [GATE CS 2020 • MCQ • 2 Marks] Let S be an array of n strings, each of length at most k , over an alphabet of constant size $|\Sigma| = 26$. What is the optimal average-case time complexity to partition these n strings into groups of anagrams using a hash table?

- (A) $\Theta(n \cdot k \log k)$
- (B) $\Theta(n \cdot k)$
- (C) $\Theta(n^2 \cdot k)$
- (D) $\Theta(k \log n)$

Answer: (B)

Solution / Explanation:

To group anagrams:

1. Two strings are anagrams if and only if their character frequency counts are identical.
2. For each string of length at most k , we compute a 26-element character frequency tuple (or count string) in $\mathcal{O}(k)$ time using counting sort or array indexing over $|\Sigma| = 26$.
3. This frequency tuple serves as the hash key in a hash map. Inserting into or looking up in the hash map takes $\mathcal{O}(k)$ time to hash the key.
4. Repeating for all n strings takes $n \times \mathcal{O}(k) = \Theta(n \cdot k)$ total time.

Sorting each string individually would take $\Theta(n \cdot k \log k)$, but character counting over fixed alphabet achieves $\Theta(n \cdot k)$.

Q6. In-Place Array Deduplication Boundary Analysis [GATE CS 2015 • NAT • 2 Marks] Consider an array A of $n = 10$ integers sorted in non-decreasing order:

$$A = [1, 1, 2, 3, 3, 3, 4, 5, 5, 6]$$

An algorithm uses two pointers `write_idx = 1` and `read_idx` from 1 to $n - 1$. When $A[\text{read_idx}] \neq A[\text{write_idx} - 1]$, it assigns $A[\text{write_idx}] = A[\text{read_idx}]$ and increments `write_idx`. What is the value of `write_idx` upon completion?

Answer: 6

Solution / Explanation:

The algorithm performs in-place deduplication of a sorted array. The distinct values present in A are:

$$\{1, 2, 3, 4, 5, 6\}$$

There are exactly 6 unique values. Since the algorithm retains the first instance of every distinct value and writes it at `write_idx`, the final value of `write_idx` equals the number of distinct elements, which is 6.

Q7. A^* Search Optimality Invariant [GATE CS 2012 / DA 2024 Scope • MSQ • 2 Marks] Which of the following conditions is/are sufficient to guarantee that A^* search finds an optimal (least-cost) path from start to goal?

- (A) Tree search is used, and the heuristic $h(n)$ is admissible.
- (B) Graph search is used with a closed set, and the heuristic $h(n)$ is consistent (monotonic).
- (C) Tree search is used, and $h(n) = 0$ for all nodes n .
- (D) Graph search is used with a closed set, and $h(n)$ is admissible but not consistent, without node re-opening.

Answer: (A, B, C)

Solution / Explanation:

- (A) is True: For A^* tree search, admissibility ($0 \leq h(n) \leq h^*(n)$) is sufficient to guarantee optimality.
- (B) is True: For A^* graph search (which never re-opens closed nodes), consistency ($h(n) \leq c(n, a, n') + h(n')$) ensures that when a node is expanded, its optimal path has already been found.
- (C) is True: Setting $h(n) = 0$ makes A^* equivalent to Uniform-Cost Search (Dijkstra's algorithm), which is optimal on graphs with non-negative edge costs.
- (D) is False: If $h(n)$ is admissible but not consistent, graph search without re-opening may lock in a suboptimal path to a node before a cheaper path to that node is explored.

Q8. A^* Goal Termination vs Generation Condition [GATE CS 2021 • MCQ • 1 Mark] In A^* search, why does the algorithm terminate only when a goal node is **selected for expansion** from the priority queue, rather than when it is first **generated** (added to the open list)?

- (A) Because the heuristic value at the goal node might not be zero.
- (B) Because a goal node can be generated along a suboptimal path before the optimal path to it has been discovered and evaluated.
- (C) Because goal nodes cannot be stored in the priority queue.
- (D) Because generating a node immediately changes its heuristic value.

Answer: (B)

Solution / Explanation:

A^* explores nodes in increasing order of $f(n) = g(n) + h(n)$. A goal node G might be generated early via a path with high cost $g_1(G)$, placing it in the priority queue. However, an unexpanded node on the frontier could lead to G via a strictly lower cost path $g_2(G) < g_1(G)$. By waiting until G is popped for expansion (i.e., when $f(G)$ is the minimum among all frontier nodes), A^* guarantees that no better path can exist.

Q9. 5-Node Graph: A^* vs Uniform Cost Search Evaluation [GATE CS 2017 • MCQ • 2 Marks]

Consider a directed graph with 5 vertices $\{S, A, B, C, G\}$ where S is the start node and G is the goal node. The edge costs are: $c(S, A) = 1$, $c(S, B) = 4$, $c(A, C) = 2$, $c(B, C) = 1$, $c(C, G) = 3$. The heuristic values to G are: $h(S) = 5$, $h(A) = 3$, $h(B) = 1$, $h(C) = 2$, $h(G) = 0$. What is the optimal path cost from S to G ?

- (A) 5
- (B) 6
- (C) 7
- (D) 8

Answer: (B)

Solution / Explanation:

Let us trace all simple directed paths from S to G :

- Path 1: $S \rightarrow A \rightarrow C \rightarrow G$. Path cost = $c(S, A) + c(A, C) + c(C, G) = 1 + 2 + 3 = 6$.
- Path 2: $S \rightarrow B \rightarrow C \rightarrow G$. Path cost = $c(S, B) + c(B, C) + c(C, G) = 4 + 1 + 3 = 8$.

The minimum path cost is $\min(6, 8) = 6$, achieved along path $S \rightarrow A \rightarrow C \rightarrow G$.

Q10. Consistent Heuristic Monotonicity Along Expansion Path [GATE CS 2016

• MCQ • 1 Mark] If a heuristic function $h(n)$ is consistent, which of the following properties holds for the sequence of evaluation function values $f(n) = g(n) + h(n)$ of nodes expanded by A^* graph search?

- (A) The sequence of f -values is monotonically non-decreasing.
- (B) The sequence of f -values is strictly decreasing.
- (C) The sequence of h -values is strictly increasing.
- (D) The sequence of g -values is constant.

Answer: (A)

Solution / Explanation:

A consistent heuristic satisfies the triangle inequality $h(n) \leq c(n, a, n') + h(n')$. Adding $g(n)$ to both sides:

$$g(n) + h(n) \leq g(n) + c(n, a, n') + h(n') = g(n') + h(n') \implies f(n) \leq f(n').$$

Therefore, the f -values along any path and the sequence of f -values of nodes selected for expansion never decrease; they are monotonically non-decreasing.

Part II: 10 GATE-Level Practice Questions

Practice Q1. Python Set Rehashing and Amortized vs Worst-Case Nuance [GATE DA Standard • MCQ • 1 Mark] In Python 3, inserting n elements one by one into an initially empty `set` takes average-case time $\mathcal{O}(n)$. What causes an individual single insertion operation to take $\Theta(n)$ time in the absence of hash collisions?

- (A) The set elements must be sorted in ascending order.
- (B) When the load factor exceeds $\frac{2}{3}$, the underlying hash table dynamically allocates a larger array and rehashes all existing entries.
- (C) Binary search is executed over the keys during insertion.
- (D) Python converts the set into a binary search tree.

Answer: (B)

Solution / Explanation:

CPython sets and dictionaries maintain a load factor threshold (typically $\approx 2/3$). When this limit is crossed, the table resizes (often quadrupling or doubling capacity). Resizing requires allocating a new bucket table and rehashing/copying all existing k elements into their new slot positions, which takes $\Theta(k) = \Theta(n)$ for that single insertion step. Amortized across all insertions, the cost per insertion remains $\mathcal{O}(1)$.

Practice Q2. Hash Collision Chain Length NAT [GATE DA Standard • NAT • 2 Marks]

A hash table with $m = 8$ buckets (indexed 0 to 7) resolves collisions using separate chaining. The hash function is $h(k) = (5k + 3) \bmod 8$. The following sequence of 7 keys is inserted:

Keys: [2, 5, 10, 13, 18, 21, 26]

After inserting all keys, what is the maximum number of keys stored in any single bucket?

Answer: 4

Solution / Explanation:

Let us compute $h(k) = (5k + 3) \bmod 8$ for each key:

- $k = 2$: $(5 \times 2 + 3) \bmod 8 = 13 \bmod 8 = 5$
- $k = 5$: $(5 \times 5 + 3) \bmod 8 = 28 \bmod 8 = 4$
- $k = 10$: $(5 \times 10 + 3) \bmod 8 = 53 \bmod 8 = 5$
- $k = 13$: $(5 \times 13 + 3) \bmod 8 = 68 \bmod 8 = 4$
- $k = 18$: $(5 \times 18 + 3) \bmod 8 = 93 \bmod 8 = 5$
- $k = 21$: $(5 \times 21 + 3) \bmod 8 = 108 \bmod 8 = 4$
- $k = 26$: $(5 \times 26 + 3) \bmod 8 = 133 \bmod 8 = 5$

Distribution across buckets:

- Bucket 5 receives keys: {2, 10, 18, 26} $\implies$ 4 keys.
- Bucket 4 receives keys: {5, 13, 21} $\implies$ 3 keys.
- All other buckets have 0 keys.

The maximum number of keys in any single bucket is 4.

Practice Q3. Hashing Pattern: Anagram Group Canonical Signature Tracing
Standard • NAT • 2 Marks] Consider a list of 8 words:

[GATE DA

```
["eat", "tea", "tan", "ate", "nat", "bat", "tab", "cat"]
```

An algorithm groups these words into anagram sets by mapping each word to a canonical tuple signature representing its character counts. What is the total number of distinct anagram groups produced?

Answer: 4

Solution / Explanation:

Let us identify the character frequencies or sorted representations for each word:

1. "eat": sorted is "aet".
2. "tea": sorted is "aet" (Group 1).
3. "tan": sorted is "ant" (Group 2).
4. "ate": sorted is "aet" (Group 1).
5. "nat": sorted is "ant" (Group 2).
6. "bat": sorted is "abt" (Group 3).
7. "tab": sorted is "abt" (Group 3).
8. "cat": sorted is "act" (Group 4).

The distinct anagram groups are:

- Group 1: {"eat", "tea", "ate"}
- Group 2: {"tan", "nat"}
- Group 3: {"bat", "tab"}
- Group 4: {"cat"}

Total number of distinct groups is 4.

Practice Q4. Deduplication of Unhashable Objects via Canonical Serialization
Standard • MCQ • 2 Marks] Suppose you are given a list of n dictionaries in Python, where each

[GATE DA

dictionary has keys "id" (int) and "data" (list of ints). You wish to eliminate duplicates such that two dictionaries are duplicates if their contents are identical. Why does calling `list(set(dict_list))` immediately fail, and what is the optimal hash-based remedy?

- (A) Dictionaries are unhashable; convert each dictionary into a tuple of sorted items, e.g., `tuple((k, tuple(v) if isinstance(v, list) else v) for k, v in sorted(d.items()))`, and store in a set.
- (B) Dictionaries cannot be compared using equality; convert keys to strings and use binary search.
- (C) Dictionaries can only be hashed if their length is prime; pad dictionaries with dummy keys.
- (D) Python sets only accept integers; cast the dictionaries to integers.

Answer: (A)

Solution / Explanation:

In Python, dictionaries and lists are mutable types that do not implement `__hash__`. Directly placing a dictionary into a set raises `TypeError: unhashable type: 'dict'`. To use $\mathcal{O}(1)$ hash set deduplication, each dictionary must be converted to an immutable, canonical representation: recursively converting lists to tuples and key-value pairs to sorted tuples of tuples ensures that identical dictionary contents produce identical hash codes and equality checks.

Practice Q5. Tracing A^* on a 5-Node Graph: Expansion Order [GATE DA Standard • MCQ • 2 Marks] Consider a directed graph with 5 vertices $\{S, A, B, C, G\}$, where S is start and G is goal.

Directed edges with step costs $c(u, v)$ are:

- $S \rightarrow A : 1, \quad S \rightarrow B : 3$
- $A \rightarrow B : 1, \quad A \rightarrow C : 4$
- $B \rightarrow C : 2, \quad B \rightarrow G : 6$
- $C \rightarrow G : 2$

The heuristic values $h(u)$ are:

$$h(S) = 4, \quad h(A) = 3, \quad h(B) = 2, \quad h(C) = 1, \quad h(G) = 0$$

A^* graph search (with consistent heuristic, no re-expansion) is run. What is the exact sequence of nodes **expanded**?

- (A) S, A, B, C, G
- (B) S, B, A, C, G
- (C) S, A, C, G
- (D) S, A, C, B, G

Answer: (A)

Solution / Explanation:

Let us trace A^* with priority queue ordered by $f(u) = g(u) + h(u)$:

- **Initialize:** $g(S) = 0, f(S) = 0 + 4 = 4$. Frontier: $\{S(4)\}$.
- **Step 1: Expand S .**
 - Successor A : $g(A) = 0 + 1 = 1, f(A) = 1 + 3 = 4$.
 - Successor B : $g(B) = 0 + 3 = 3, f(B) = 3 + 2 = 5$.Frontier: $\{A(4), B(5)\}$. Closed: $\{S\}$.
- **Step 2: Expand A ($f(A) = 4$ is min).**
 - Successor B : via A , $g(B) = 1 + 1 = 2 < 3$. Cheaper! Update $g(B) = 2, f(B) = 2 + 2 = 4$.
 - Successor C : $g(C) = 1 + 4 = 5, f(C) = 5 + 1 = 6$.Frontier: $\{B(4), C(6)\}$. Closed: $\{S, A\}$.
- **Step 3: Expand B ($f(B) = 4$ is min).**
 - Successor C : via B , $g(C) = 2 + 2 = 4 < 5$. Cheaper! Update $g(C) = 4, f(C) = 4 + 1 = 5$.
 - Successor G : $g(G) = 2 + 6 = 8, f(G) = 8 + 0 = 8$.Frontier: $\{C(5), G(8)\}$. Closed: $\{S, A, B\}$.
- **Step 4: Expand C ($f(C) = 5$ is min).**
 - Successor G : via C , $g(G) = 4 + 2 = 6 < 8$. Cheaper! Update $g(G) = 6, f(G) = 6 + 0 = 6$.Frontier: $\{G(6)\}$. Closed: $\{S, A, B, C\}$.
- **Step 5: Expand G ($f(G) = 6$ is min). Goal reached.**

The exact expansion sequence is S, A, B, C, G .

Practice Q6. Tracing A^* on 5-Node Graph: Optimal Cost Value [GATE DA Standard • NAT • 1 Mark] For the 5-node graph given in Practice Q5, what is the cost $g(G)$ of the optimal path found by A^* when the goal G is expanded?

Answer: 6

Solution / Explanation:

From the trace in Practice Q5:

- Path found: $S \rightarrow A \rightarrow B \rightarrow C \rightarrow G$.
- Step costs: $c(S, A) = 1, c(A, B) = 1, c(B, C) = 2, c(C, G) = 2$.
- Total cost $g(G) = 1 + 1 + 2 + 2 = 6$.

Practice Q7. Subarray Deduplication with Sliding Window Hash Set [GATE DA Standard • MCQ • 2 Marks] We are given an array of n integers and a positive window size $k \leq n$. We want to find whether there exists any contiguous subarray of length k that contains duplicate values. Which approach provides the optimal time and space complexity?

- (A) For each of the $(n - k + 1)$ windows, sort the k elements in $\mathcal{O}(k \log k)$; Time $\mathcal{O}(nk \log k)$, Space $\mathcal{O}(k)$.
- (B) Maintain a sliding hash set of at most k elements: insert incoming element and delete the

element leaving the window ($i - k$); Time $\mathcal{O}(n)$ average, Space $\mathcal{O}(k)$.

(C) Insert all n elements into a single global hash set; Time $\mathcal{O}(n)$, Space $\mathcal{O}(n)$.

(D) Compare all pairs within every window; Time $\mathcal{O}(nk^2)$, Space $\mathcal{O}(1)$.

Answer: (B)

Solution / Explanation:

Using a sliding window with a hash set:

- The set contains only elements within the current active window of size $\leq k$.
- For each index i from 0 to $n - 1$:
 1. If $A[i]$ already exists in the set, a duplicate within window distance k is found.
 2. Insert $A[i]$ into the set.
 3. If $i \geq k$, remove $A[i - k]$ from the set to maintain the window invariant.
- Each element is added once and removed at most once $\implies \mathcal{O}(1)$ average time per element.
- Overall average time is $\mathcal{O}(n)$, and auxiliary space is strictly bounded by the window size $\mathcal{O}(k)$.

Practice Q8. Monotonicity of f -Values Across Expansions in 5-Node Graph

[GATE DA

Standard • MSQ • 2 Marks] Refer to the 5-node graph search in Practice Q5 with expansion order S, A, B, C, G . Which of the following statements is/are **TRUE**?

- (A) The heuristic h is consistent across every directed edge in the graph.
- (B) The f -values of the expanded nodes in order are 4, 4, 4, 5, 6.
- (C) No node was expanded more than once.
- (D) If $h(C)$ were changed from 1 to 4, the heuristic would remain admissible.

Answer: (A, B, C)

Solution / Explanation:

- Check consistency $h(u) \leq c(u, v) + h(v)$:

- $S \rightarrow A$: $4 \leq 1 + 3 = 4$ (Holds)
- $S \rightarrow B$: $4 \leq 3 + 2 = 5$ (Holds)
- $A \rightarrow B$: $3 \leq 1 + 2 = 3$ (Holds)
- $A \rightarrow C$: $3 \leq 4 + 1 = 5$ (Holds)
- $B \rightarrow C$: $2 \leq 2 + 1 = 3$ (Holds)
- $B \rightarrow G$: $2 \leq 6 + 0 = 6$ (Holds)
- $C \rightarrow G$: $1 \leq 2 + 0 = 2$ (Holds)

Thus, h is consistent (A is True).

- f -values at expansion: $f(S) = 4, f(A) = 4, f(B) = 4, f(C) = 5, f(G) = 6$. The sequence is 4, 4, 4, 5, 6, which is monotonically non-decreasing (B is True).
- Because h is consistent, each node is expanded at most once (C is True).
- True cost from C to G is $c(C, G) = 2$. If $h(C) = 4$, then $h(C) > h^*(C) = 2$, which violates admissibility (D is False).

Practice Q9. Degraded Hash Collision Impact on Membership Testing [GATE DA Standard

• MCQ • 1 Mark] A student implements a custom class and mistakenly defines `__hash__(self)` to return 0 for every instance, while implementing correct value-based equality in `__eq__`. If n distinct instances of this class are added to a Python set, what will be the time taken to check membership of an arbitrary element?

- (A) $\Theta(1)$
- (B) $\Theta(\log n)$
- (C) $\Theta(n)$
- (D) $\Theta(n \log n)$

Answer: (C)

Solution / Explanation:

When every object produces the hash value 0:

- All n objects map to the exact same hash bucket / initial probing slot.
- To determine whether an element is present, Python's open-addressing lookup must resolve all collisions along the probe sequence, calling `__eq__` on every stored item until either an equal object or an unused slot is found.
- Thus, membership testing degrades from average-case $\mathcal{O}(1)$ to worst-case $\Theta(n)$ linear scan.

Practice Q10. A^* Search Open List Re-parenting

[GATE DA

Standard • MSQ • 2 Marks] During the execution of A^* graph search, a node v that is currently in the frontier (open list) is reached again via a newly explored edge from u . Which of the following actions is/are executed by the algorithm?

- (A) If $g(u) + c(u, v) < g(v)$, the value of $g(v)$ is updated to $g(u) + c(u, v)$.

- (B) If $g(u) + c(u, v) < g(v)$, the parent pointer of v is updated to point to u .
- (C) The priority of v in the open list priority queue is updated to the smaller value $f(v) = g(u) + c(u, v) + h(v)$.
- (D) If $g(u) + c(u, v) \geq g(v)$, the new path is discarded and v remains unchanged.

Answer: (A, B, C, D)

Solution / Explanation:

All statements accurately describe the standard relaxation and re-parenting step of A^* search:

- If the new path through u offers a strictly lower path cost g , $g(v)$ and $\text{parent}(v)$ are updated to reflect the superior route (A and B are True).
- Because $g(v)$ decreases, $f(v) = g(v) + h(v)$ decreases, requiring a decrease-key operation in the priority queue (C is True).
- If the newly discovered path is equal or more expensive, it is simply discarded without altering the frontier (D is True).

End of Question Bank — GATE Data Science & Artificial Intelligence (DA)